Columbia University
Department of Industrial Engineering and Operations Research

IEOR E4540: Data Mining

Project 3

Improving efficient Vision Transformers with Relative Positional Encodings (RPEs)

Thomas Flamand (UNI: tf2636)

Instructor: Dr. Krzysztof Choromanski

Fall, 2025

Contents

Abstract

This project aims to improve the performance of efficient Vision Transformers by integrating Relative Positional Encodings (RPE). We build a lightweight Vision Transformer architecture based on the efficient Performer attention mechanism, considering two Performer variants: (a) leveraging positive random features for an unbiased approximation of the softmax kernel, and (b) Performer-ReLU. We then enhance these Performer-ViT models with three recent RPE mechanisms:

1) General RPE scheme [Luo et al., 2021]

2) Circulant-STRING mechanism [Schenck et al., 2025]

3) RoPE rotary encoding [Su et al., 2024]

The objective is to evaluate to what extent these RPE modules can improve the accuracy of lightweight ViT models, especially on resource-constrained devices, and whether they can close the accuracy gap between a standard full-attention ViT and Performer-based variants. We provide efficient implementations of all three enriched Performer architectures and compare them against brute-force attention ViTs on MNIST and CIFAR-10, analyzing not only classification accuracy but also training and inference time. Ultimately, this project explores whether combining Performers with RPEs can preserve scalability while improving representational power.

1 Introduction

Transformers [1] rely on the self-attention mechanism to model interactions between elements of a sequence. Given a query–key–value triplet (Q, K, V), self-attention is defined as:

$$\mathrm{Att}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{1}$$

where d_k denotes the key dimensionality. While highly expressive, this formulation requires forming a full $N \times N$ similarity matrix, resulting in a time and memory complexity of $\mathcal{O}(N^2)$, which becomes prohibitive for long sequences or high-resolution vision inputs.

To alleviate this limitation, linear attention mechanisms have been proposed. In particular, the Performer [3] reformulates self-attention using kernel feature mappings. The key idea is to approximate the softmax kernel by a feature map $\phi(\cdot)$ such that:

$$\exp(QK^\top) \;\approx\; \phi(Q)\phi(K)^\top. \tag{2}$$

This allows attention to be computed as:

$$\mathrm{Att}(Q, K, V) \;\approx\; \phi(Q)\big(\phi(K)^\top V\big)\big(\phi(Q)\phi(K)^\top \mathbf{1}\big)^{-1}, \tag{3}$$

which avoids constructing the $N \times N$ matrix and reduces the attention complexity to $\mathcal{O}(N)$ when the feature dimension is fixed.

Performer-based Vision Transformers retain the overall architecture of standard ViTs while offering

significantly improved scalability. However, replacing exact softmax attention with kernel-based approximations can reduce representational capacity and lead to a drop in downstream accuracy compared to full-attention models.

Positional information plays a critical role in mitigating this loss. Since self-attention is inherently permutation-invariant, positional encodings are required to capture structural relationships in data. Relative positional encodings (RPEs) encode distances between tokens directly within the attention mechanism, providing stronger inductive biases than absolute positional encodings. When combined with linear attention, RPEs can enrich token interactions and help recover part of the accuracy lost due to approximation.

This project investigates whether integrating relative positional encodings into Performer-based Vision Transformers can improve downstream accuracy while preserving the linear-time complexity of Performer attention. We evaluate Performer-ViT models with and without RPEs on MNIST and CIFAR-10, and compare them to standard ViTs in terms of classification accuracy, training stability, and inference time. Our goal is to assess whether RPE-augmented Performer architectures can close the accuracy gap with full-attention models while maintaining favorable computational properties.

2 Methodology

2.1 Lightweight Performer-ViT Architecture

Our base model is a lightweight Vision Transformer built upon Performer linear attention. The overall architecture follows the standard ViT pipeline, with reduced dimensionality to match the scale of MNIST and CIFAR-10. An input image is first divided into non-overlapping patches (e.g., 4×4 pixel patches for CIFAR-10). Each patch is flattened and linearly projected into a d-dimensional embedding space, forming a sequence of patch embeddings. A trainable [CLS] token is prepended to the sequence, as in [2], and serves as a global representation for classification.

Formally, given an image $x \in \mathbb{R}^{H \times W \times C}$ and patch size P, the number of patches is:

$$N = \frac{HW}{P^2},$$

and each patch x_i is projected via a trainable matrix $W_p \in \mathbb{R}^{(P^2C) \times d}$:

$$z_i = x_i W_p.$$

The model then stacks L Transformer blocks. Each block consists of two sub-layers: (1) a multi-head self-attention layer followed by a residual connection and layer normalization, and (2) a position-wise feed-forward network (MLP) with GeLU activation, also followed by a residual connection and normalization.

In a conventional Transformer, attention is computed as:

$$\mathrm{Att}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \tag{4}$$

which requires forming an $N \times N$ attention matrix. This results in a time and memory complexity of:

$$\mathcal{O}(N^2 d),$$

which becomes impractical for large sequences or high-resolution images.

To address this limitation, we replace softmax attention with the Performer mechanism [3]. Performer rewrites attention using kernel feature maps, relying on the identity:

$$\exp(QK^\top) \;\approx\; \phi(Q)\,\phi(K)^\top, \tag{5}$$

where $\phi : \mathbb{R}^d \to \mathbb{R}^m$ is a feature map satisfying:

$$\mathbb{E}\Big[\phi(Q)\phi(K)^\top\Big] = \exp(QK^\top).$$

This approximation allows attention to be computed in linear form:

$$\mathrm{Att}(Q, K, V) \;\approx\; \phi(Q)\left(\phi(K)^\top V\right)\left(\phi(Q)\phi(K)^\top \mathbf{1}\right)^{-1}, \tag{6}$$

avoiding the explicit construction of the $N \times N$ matrix and reducing complexity to:

$$\mathcal{O}(Ndm),$$

which is linear in the sequence length N when the number of features m is fixed.

In this work, we consider two Performer variants:

The FAVOR+ variant [3] approximates the softmax kernel using positive random features. A valid feature map is:

$$\phi(x) = \frac{1}{\sqrt{m}} \exp\left(-\frac{\|x\|^2}{2}\right) \exp(Wx), \tag{7}$$

where $W \in \mathbb{R}^{m \times d}$ contains i.i.d. Gaussian entries. The exponential form ensures non-negativity, which is required for the normalization in Equation 6. FAVOR+ provides an unbiased estimator of the softmax attention kernel and serves as the main Performer baseline in our experiments.

In addition to FAVOR+, we also evaluate the Performer-ReLU variant, where the kernel feature map is defined as:

$$\phi(x) = \mathrm{ReLU}(x).$$

This choice removes the need for random projections and yields a deterministic linear attention mechanism. While Performer-ReLU does not approximate the softmax kernel explicitly, it pre-

serves the linear attention structure and often exhibits improved numerical stability. We include this variant to assess the trade-off between approximation fidelity (FAVOR+) and simplicity and stability (ReLU-based attention).

All other architectural components (layer normalization, residual connections, and MLP blocks) remain identical across models. This ensures that differences in performance can be attributed primarily to the attention mechanism and the incorporation of relative positional encodings, which we describe next.

2.2 Relative Positional Encodings Considered

We integrate and compare three relative positional encoding (RPE) mechanisms from the literature, representative of different approaches: (a) a general RPE scheme based on learned biases (Luo et al., 2021), (b) the circulant-STRING encoding proposed by Schenck et al. (2025), and (c) the rotary encoding RoPE introduced by Su et al. (2024). We describe the mathematical principle of each module below.

2.2.1 General RPE (Luo et al., 2021)

The most general form of relative positional encoding consists in introducing a bias dependent on the distance between two positions, which modifies the attention computation. If Q_i and K_j denote the query and key vectors at positions i and j, a relative positional encoding adds a learned term b_{i-j} to the attention score:

$$e_{ij} \;=\; \frac{Q_i K_j^\top + b_{\,i-j}}{\sqrt{d}}\,, \tag{8}$$

where b_k is a learnable parameter associated with offset $k = i - j$. This formulation allows the model to modulate interactions based on relative distance, capturing the relative order of patches rather than their absolute positions.

While this additive bias is directly compatible with softmax attention, it cannot be naively incorporated into linear attention without breaking the kernel factorization. In the Performer setting, the relative bias does not aim to approximate the exponential kernel itself; instead, it acts as a structured modulation applied along the sequence dimension. As shown by Luo et al. [4], the bias matrix induced by b_{i-j} has a Toeplitz structure, which enables efficient application via convolution. This operation can be integrated after the kernel-based projection step, preserving the linear complexity of Performer attention while injecting relative positional information. Consequently, the general RPE acts as a complementary positional correction rather than as part of the kernel approximation.

Formally, a Toeplitz matrix $B \in \mathbb{R}^{N\times N}$ satisfies:

$$B_{ij} = B_{i+1,j+1},$$

and its matrix-vector product can be written as a convolution:

$$(Bx)_i = \sum_{k=-(N-1)}^{N-1} b_k \, x_{i-k},$$

which can be computed using the Fast Fourier Transform in:

$$O(N \log N),$$

instead of $O(N^2)$ for a naïve implementation.

2.2.2 Circulant-STRING Encoding (Schenck et al., 2025)

The STRING mechanism (Skew-symmetric Transformations for Rotary Invariant Gradients) extends the principle of rotary position encoding to multi-dimensional data (such as 2D images or 3D scenes) using Lie algebra. *Schenck et al.* [5] propose a variant called circulant-STRING , in which the rotation generator matrices are constrained to be circulant and anti-symmetric. Intuitively, this means defining a matrix S (of size $d \times d$) such that S is circulant (each row is a circular shift of the previous) and $S^\top = -S$ (skew symmetry). Such a matrix S can be parameterized with a small number of independent parameters while representing a rotation operation in representation space.

A circulant matrix $C \in \mathbb{R}^{d \times d}$ is defined by its first row $(c_0, c_1, \ldots, c_{d-1})$ such that:

$$C_{ij} = c_{(j-i) \bmod d}.$$

Circulant matrices admit a closed-form eigendecomposition:

$$C = F^* \Lambda F,$$

where F is the discrete Fourier transform (DFT) matrix and Λ is diagonal. As a consequence,

$$\exp(C) = F^* \exp(\Lambda) F,$$

meaning that the matrix exponential reduces to element-wise exponentiation in the Fourier domain.

Because $S^\top = -S$, the eigenvalues of S are purely imaginary, which guarantees that:

$$R = \exp(S\delta)$$

is an orthogonal rotation matrix preserving vector norms:

$$\|Rx\| = \|x\|.$$

In circulant-STRING, the spatial coordinates (u, v) of an image patch are associated with a rotational transformation $\exp(S \cdot \delta_{uv})$, where δ_{uv} is a vector or scalar representing the relative displacement. The circulant structure of S enables the use of the Fourier transform to diagonalize S, making the computation of $\exp(S \cdot \delta)$ highly efficient (the product with R_{uv} can be computed via FFTs over dimensions, greatly reducing the cost compared to arbitrary rotations). In practice, circulant-STRING jointly encodes relative displacements along two axes (horizontal and vertical) within a single linear operation. This mechanism has shown significant improvements in capturing 2D relative position, outperforming absolute encodings and approaching the performance of more complex variants (such as Cayley-parameterized STRING) while remaining simpler to implement [5].

2.2.3 Rotary Encoding RoPE (Su et al., 2024)

The Rotary Position Embedding (RoPE) introduced by *Su et al.* [6] provides an elegant way to incorporate relative positional information multiplicatively. Rather than adding a bias to the attention score, RoPE directly multiplies (rotates) the Q and K vectors by a rotation matrix that depends on the absolute position of the patch. Thus, for a patch at position i, its representations Q_i and K_i are transformed into $Q'_i = R_{\theta,i} Q_i$ and $K'_i = R_{\theta,i} K_i$, where $R_{\theta,i}$ is a rotation matrix with angle proportional to i. Using rotation matrices defined differently for each dimension or pair of dimensions in the latent space leads to a final dot product $Q'_i \cdot K'_j$ that depends only on the difference $(i - j)$ of the positions, and not on the absolute values i and j.

Concretely, RoPE operates by dividing each attention head vector into 2D subspaces and applying increasingly large rotations for higher dimensions. For example, in a two-dimensional subspace, the rotation matrix is:

$$R_{\theta,t} = \begin{pmatrix} \cos(t\,\theta) & -\sin(t\,\theta) \\ \sin(t\,\theta) & \cos(t\,\theta) \end{pmatrix}, \tag{9}$$

where t denotes the (absolute) positional index and θ is a fixed constant determining the rotation rate per index step. Each component pair $(2k, 2k+1)$ of the vector Q_i is thus treated as coordinates (x, y) that are rotated by an angle $i \cdot \theta$ (and similarly for K_i).

RoPE can be expressed equivalently using complex numbers. If we represent a 2D component as $q_i^{(k)} = x + iy$, then:

$$q_i'^{(k)} = q_i^{(k)}\, e^{i\,\theta\,i}, \qquad k_j'^{(k)} = k_j^{(k)}\, e^{i\,\theta\,j}.$$

Their interaction becomes:

$$q_i'^{(k)} \big(k_j'^{(k)}\big)^* = q_i^{(k)} \big(k_j^{(k)}\big)^*\, e^{i\theta(i-j)},$$

which depends only on the relative offset $(i - j)$. This complex formulation makes explicit that RoPE introduces a relative phase term without changing vector magnitudes.

Expanding the rotated dot product yields:

$$Q'_i \cdot K'_j = \sum_k \Big[\cos((i-j)\theta)\,(Q^{(k)}_{i,1}K^{(k)}_{j,1} + Q^{(k)}_{i,2}K^{(k)}_{j,2}) + \sin((i-j)\theta)\,(Q^{(k)}_{i,1}K^{(k)}_{j,2} - Q^{(k)}_{i,2}K^{(k)}_{j,1}) \Big], \quad (10)$$

showing that the positional dependence is purely a function of $(i-j)$.

An important property of RoPE is that $R_{\theta,t}$ is orthogonal:

$$R^\top_{\theta,t} R_{\theta,t} = I, \qquad \|R_{\theta,t}x\| = \|x\|,$$

which guarantees that applying RoPE does not change the norm of Q or K, preserving the scale of attention logits.

RoPE is appealing due to its simple integration (requiring only sinusoidal vector operations) and does not introduce additional learned parameters (unlike the general RPE bias formulation). Despite its simple form, RoPE has been shown to improve Transformer performance on a wide range of tasks by enabling better extrapolation to longer sequences [6].

2.3 Integrating RPEs into Performer Attention

Each RPE mechanism described above is integrated into the model at the Performer-ViT attention layer. Concretely, we adapt the multi-head attention computation to incorporate relative positional information in a manner compatible with the linear Performer approach.

For the general learned-bias RPE (Equation 8), we store a bias vector b_k for each possible offset k within the patch sequence (size $2N-1$ if N patches). In the Performer setting, this relative bias is not directly added to the dot-product $QK^\top$, as this would break the kernel factorization required for linear attention. Instead, following Luo et al. [4], the bias is applied as a structured modulation along the sequence dimension after the kernel-based projection. Exploiting the Toeplitz structure of the bias matrix, this operation can be implemented efficiently as a convolution via the Fast Fourier Transform (FFT), resulting in an additional cost of $O(N \log N)$. The inclusion of such relative biases acts as an informative regularization term that has been shown to stabilize training in linear Transformers.

For rotation-based encodings (circulant-STRING and RoPE), integration is relatively straightforward. Before computing linear attention, we apply to each query Q_i and key K_i the transformation defined by position i. In the case of RoPE, this amounts to multiplying (Q_i, K_i) by rotation matrices $R_{\theta,i}$ (defined in Equation 9) within each two-dimensional subspace. Because $R_{\theta,i}$ is orthogonal, this transformation preserves vector norms:

$$\|Q'_i\| = \|Q_i\|, \qquad \|K'_i\| = \|K_i\|,$$

ensuring that the scale of attention logits remains unchanged.

Crucially, applying RoPE before the Performer feature map preserves the statistical properties required by the FAVOR+ kernel approximation. The Performer estimator relies on random projections Wx with i.i.d. Gaussian entries. Since the Gaussian distribution is rotationally invariant, applying a rotation to Q and K does not alter the distribution of the projected features, ensuring that the kernel approximation remains valid.

For circulant-STRING, we compute for each position i the matrix $R_i = \exp(S \cdot c_i)$, where c_i represents the spatial coordinates of the patch. Because S is circulant and skew-symmetric, R_i is orthogonal and can be computed efficiently via FFT. These rotation operations occur prior to the Performer attention computation, so the random feature approximation naturally incorporates the positional effect. In both cases, the addition of these encodings does not increase the asymptotic complexity of the attention layer. The rest of the model architecture (normalization, MLP, etc.) remains unchanged, except that absolute positional encodings are replaced by these relative mechanisms.

3 Experiments

3.1 Datasets and Experimental Setup

We evaluate all model variants on two standard vision benchmarks: MNIST and CIFAR-10. MNIST consists of 28×28 grayscale images of handwritten digits from 10 classes. Following common practice for Vision Transformers, images are resized to 32×32 before patchification. CIFAR-10 contains 32×32 RGB images from 10 object categories. Both datasets provide 50,000 training images and 10,000 test images; we use the standard train/test splits without a separate validation set.

All models are trained in a supervised classification setting using the cross-entropy loss. We use the Adam optimizer with a fixed learning rate of 10^{-3} and a batch size of 128. For MNIST, models are trained for 5 epochs; for CIFAR-10, we also train for 5 epochs in order to keep computational cost manageable and ensure a consistent comparison across settings. For CIFAR-10, we apply light data augmentation during training in the form of random horizontal flips, while no augmentation is used for MNIST.

All models are trained from scratch with identical initialization and optimization settings. No pretrained weights are used. To ensure a fair comparison between standard ViT and Performer-based architectures, all models share the same architectural hyperparameters, and only the attention mechanism and relative positional encoding (RPE) vary.

Importantly, we deliberately do not use weight decay in our optimization setup. While weight decay (e.g., AdamW) is commonly employed in Vision Transformers to improve absolute performance, our goal in this work is not to maximize benchmark accuracy but to isolate the effect of relative positional encodings within Performer-based attention mechanisms. Introducing additional regularization could interact with positional inductive biases and confound the attribution

of performance differences. By using a simple Adam optimizer without weight decay across all experiments, we ensure that observed performance variations can be primarily attributed to the attention approximation and the choice of RPE.

We adopt a compact Vision Transformer configuration suitable for small-scale vision datasets. Input images of size 32×32 are divided into non-overlapping patches of size 4×4, yielding 64 patch tokens. A learnable [CLS] token is prepended to the sequence, resulting in a total sequence length of 65 tokens. Patch embeddings are obtained via a convolutional projection layer, followed by stacking Transformer encoder blocks.

The model depth is fixed to $L = 4$, with each block consisting of a pre-layer normalization, a multi-head self-attention layer, and a feed-forward MLP. The embedding dimension is set to $d = 64$, and the number of attention heads to $h = 4$, resulting in a head dimension of 16. The MLP expansion ratio is fixed to 4, following the standard Vision Transformer design.

We compare three attention mechanisms: (i) standard full softmax attention, (ii) Performer FAVOR+ attention using positive random features to approximate the softmax kernel, and (iii) Performer attention with a ReLU feature map. For FAVOR+, we use $m = 64$ random features per head, which is sufficient given the short sequence length induced by image patching.

Relative positional encodings are only applied to Performer-based models. We consider three RPE variants: rotary positional embeddings (RoPE), a classic multiplicative RPE adapted to Performer attention, and a circulant STRING-based RPE. When RoPE is used, it is applied directly to the query and key projections before kernel feature mapping. The classic and STRING RPEs are applied as multiplicative weighting matrices along the sequence dimension.

We report classification accuracy on the test set for all models. In addition, we measure the total inference time over the test set and report the average inference time per image. All experiments are conducted on a GPU, and inference timings are therefore indicative rather than absolute. Given the relatively short sequence lengths in MNIST and CIFAR-10, efficiency gains from Performer attention are modest but observable.

3.2 Results and Analysis

We evaluate all models using test accuracy at the final training epoch. Beyond final accuracy, we analyze convergence behavior, generalization, and computational efficiency using accuracy curves, train–test comparisons, and inference-time measurements.

Table 1 summarizes the final test accuracy achieved by all models on MNIST and CIFAR-10. On MNIST, all models converge rapidly within five training epochs. The standard ViT reaches high accuracy, while Performer-based models without relative positional encoding (RPE) perform noticeably worse. Introducing RPEs substantially improves Performer performance: both RoPE and circulant-STRING achieve accuracies above 95% on MNIST, slightly surpassing the full-attention ViT.

Model	**MNIST** Accuracy (%)	**CIFAR-10** Accuracy (%)
ViT (Full Attention)	87.56	56.93
Performer-FAVOR+ (no RPE)	86.62	55.24
Performer-FAVOR+ + RPE (Classic)	86.71	54.39
Performer-FAVOR+ + RPE (RoPE)	97.44	62.70
Performer-FAVOR+ + RPE (circulant-STRING)	95.76	67.30
Performer-ReLU (no RPE)	88.99	54.49
Performer-ReLU + RPE (Classic)	75.66	30.60
Performer-ReLU + RPE (RoPE)	96.97	62.02
Performer-ReLU + RPE (circulant-STRING)	95.85	69.43

Table 1: Final test accuracy (%) at the last training epoch (5 for MNIST, 20 for CIFAR-10) for standard ViT and Performer-ViT models with and without relative positional encodings.

This behavior is consistent with the accuracy curves, which show faster and smoother convergence when relative positional information is injected into the attention mechanism.

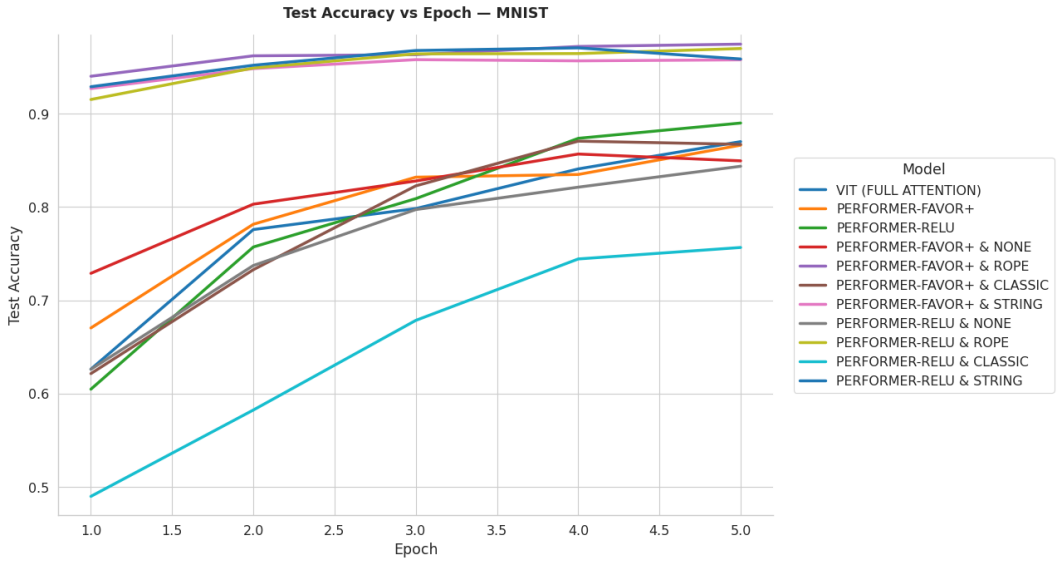

Figure 1: Test accuracy versus epoch on MNIST. All models converge rapidly, with RPE-equipped Performer models achieving faster and more stable convergence than their no-RPE counterparts.

On CIFAR-10, the task is more challenging and models are trained for 20 epochs due to computational constraints. The standard ViT reaches 56.93% accuracy at epoch 20. Performer-ViT models without RPE underperform this baseline, confirming that linear attention approximations slow down optimization in early training. However, incorporating RPEs leads to consistent improvements. RoPE increases accuracy to approximately 62%, while circulant-STRING yields the best Performer results, reaching around 67% for both FAVOR+ and ReLU attention.

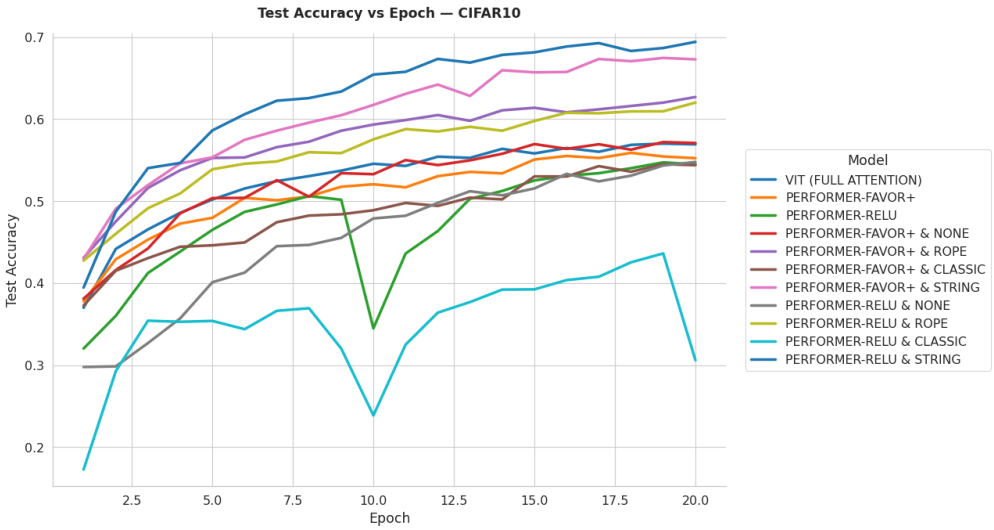

Figure 2: Test accuracy versus epoch on CIFAR-10. RPE-enhanced Performer models converge faster and achieve higher accuracy than no-RPE variants, substantially reducing the gap with the full-attention ViT.

Train–test accuracy comparisons highlight the impact of RPEs primarily on optimization stability rather than on a systematic reduction of the train–test gap. Across both FAVOR+ and ReLU variants, models trained without RPE exhibit noisier test curves and less reliable convergence, especially on CIFAR-10. RoPE and circulant-STRING consistently stabilize training dynamics, leading to smoother test accuracy trajectories and higher final performance. While the absolute train–test gap is not consistently reduced across all settings, RPEs mitigate optimization pathologies and improve convergence behavior, which indirectly benefits generalization.

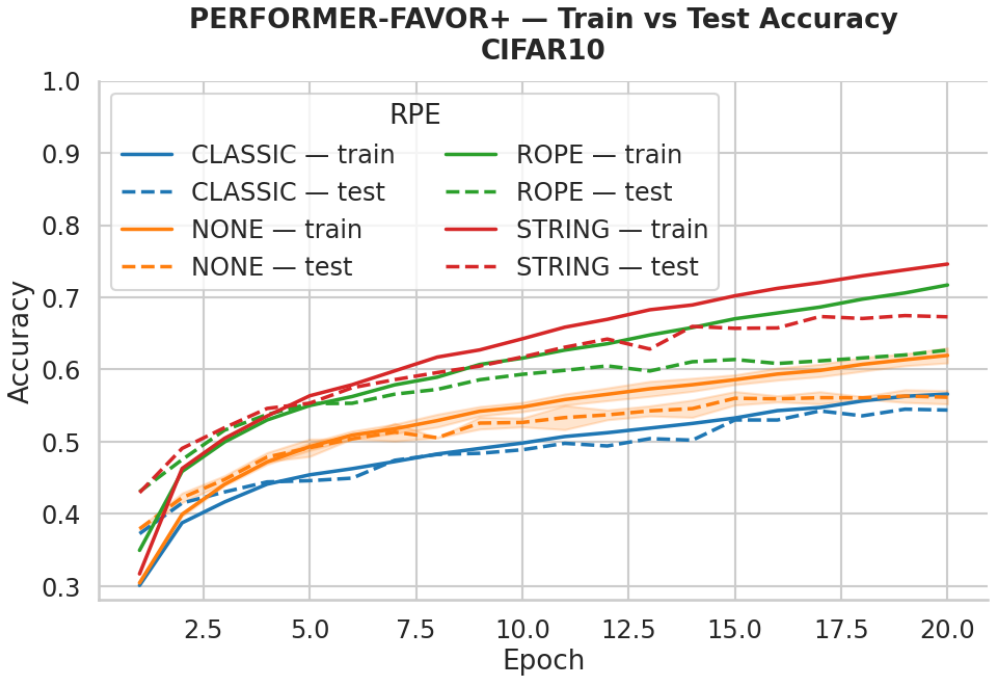

Figure 3: Training versus test accuracy for Performer-FAVOR+ on CIFAR-10. RPE mechanisms improve convergence stability or final test accuracy, although the train–test gap is not systematically reduced.

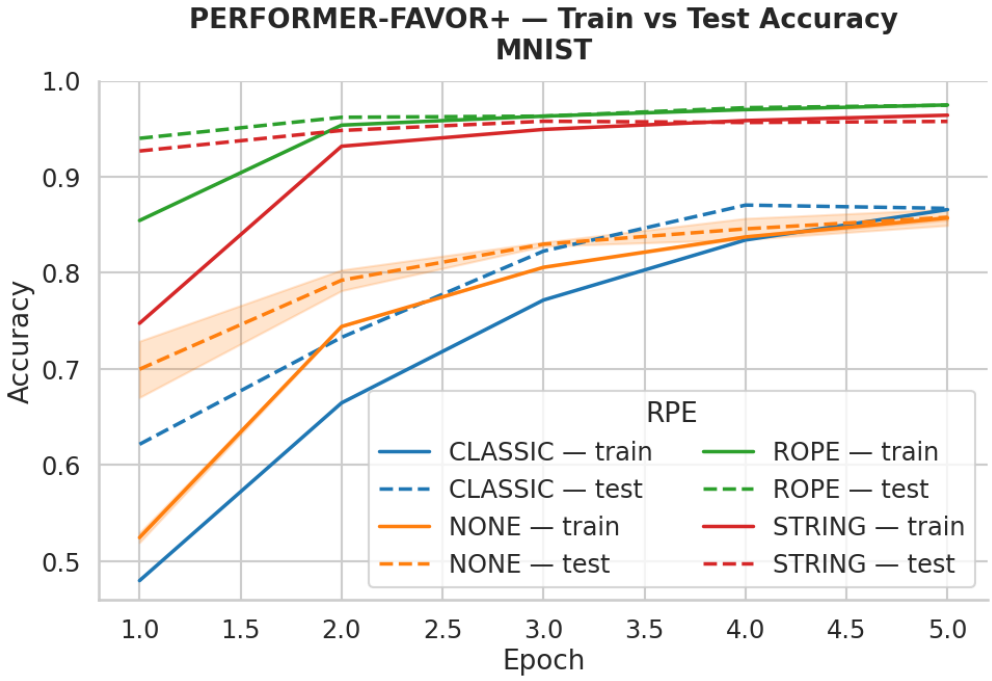

Figure 4: Training versus test accuracy for Performer-FAVOR+ on MNIST. All models converge rapidly, with RoPE and circulant-STRING significantly improving both final accuracy and convergence's speed.

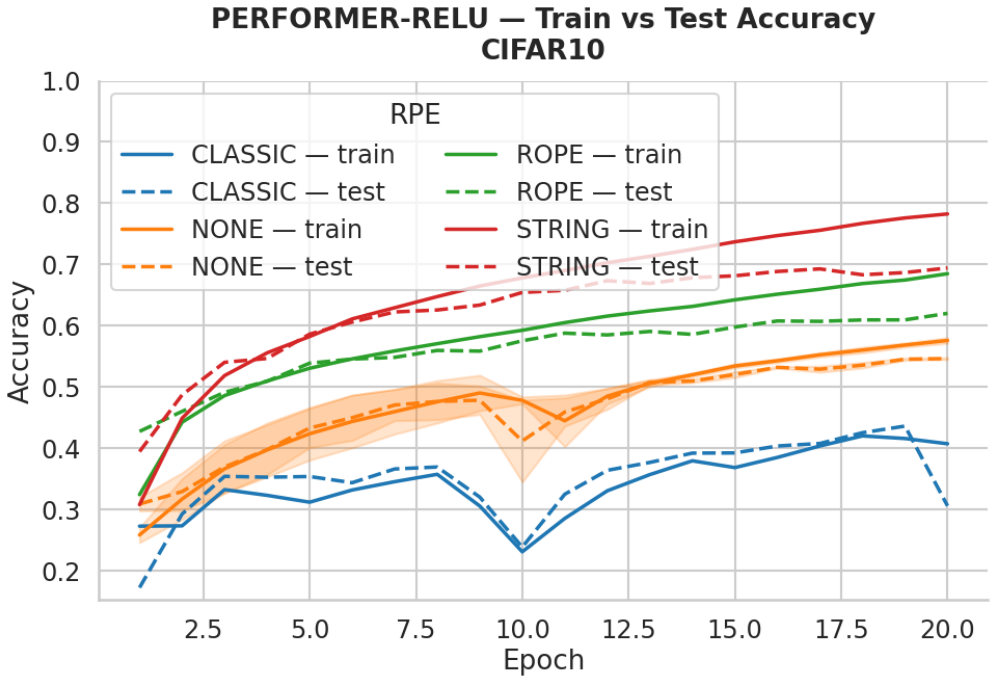

Figure 5: Training versus test accuracy for Performer-ReLU on CIFAR-10. Without RPE (or with the 'most general RPE'), training exhibits severe instability and abrupt drops in test accuracy, while RoPE and circulant-STRING stabilize optimization and yield smoother test trajectories.

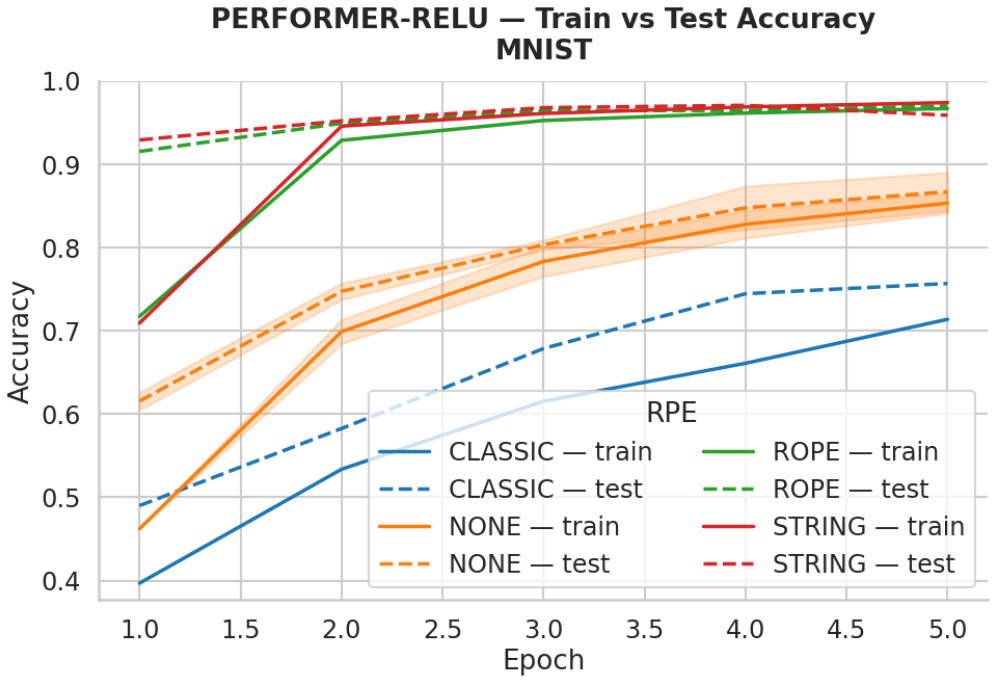

Figure 6: Training versus test accuracy for Performer-ReLU on MNIST. RoPE and circulant-STRING lead to faster convergence and smaller train-test gap. Classic RPE yields to much more modest effects and even perform worse than without RPE.

Notably, the stabilizing effect of RPEs is most pronounced for the ReLU-based Performer on CIFAR-10. In this setting, the absence of positional information leads to highly unstable optimization and abrupt performance degradation, whereas RoPE and circulant-STRING enable smooth and reliable training dynamics. This highlights the importance of relative positional encodings for stabilizing linear attention mechanisms under challenging optimization conditions.

We measure inference time as the total evaluation time per epoch on the test set. Performer-based models generally achieve lower inference times than the standard ViT, reflecting the linear complexity of Performer attention. Relative positional encodings introduce additional computational overhead, particularly for the FFT-based circulant-STRING mechanism. However, as shown in Fig. 7 and Fig. 8, these costs are compensated by substantial gains in test accuracy, yielding favorable accuracy–inference efficiency trade-offs, especially on CIFAR-10.

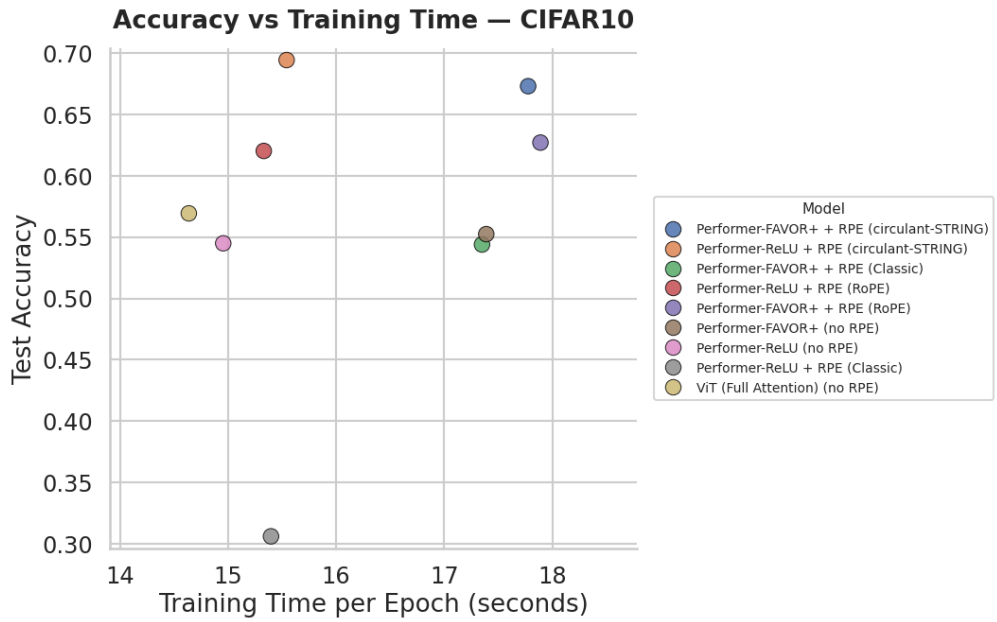

Figure 7: Test accuracy versus inference time per epoch on CIFAR-10. RPE-equipped Performer models achieve superior accuracy–efficiency trade-offs compared to no-RPE variants and the full-attention ViT.

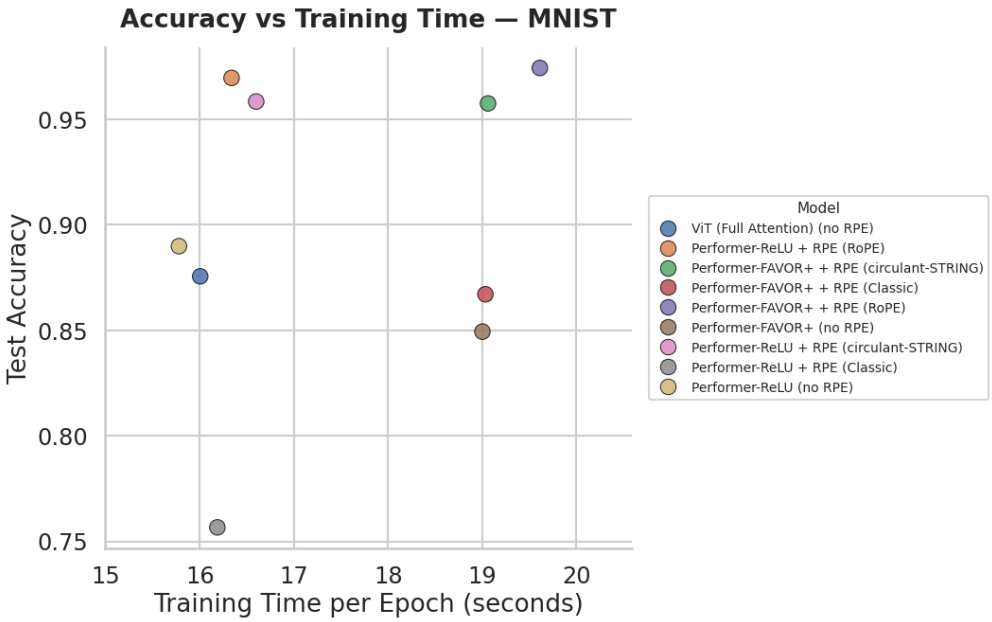

Figure 8: Test accuracy versus inference time per epoch on MNIST. Due to short sequences, inference-time differences remain limited, while RPE-equipped models still achieve higher accuracy.

To further assess computational efficiency, we analyze the inference cost of all models by measuring the average inference time per epoch on the test set. As shown in Fig. 9 and Fig. 10, the full-attention ViT remains among the fastest models in terms of inference time for the considered sequence length. Performer-based models exhibit inference times that are comparable to, and in some cases slightly higher than, the ViT, despite their linear attention formulation. Importantly, these differences remain moderate in absolute terms, particularly due to the short sequence length (65 tokens). Despite this additional cost, RPE-equipped Performer models achieve substantially higher test accuracy, especially on CIFAR-10, resulting in favorable accuracy–inference efficiency trade-offs.

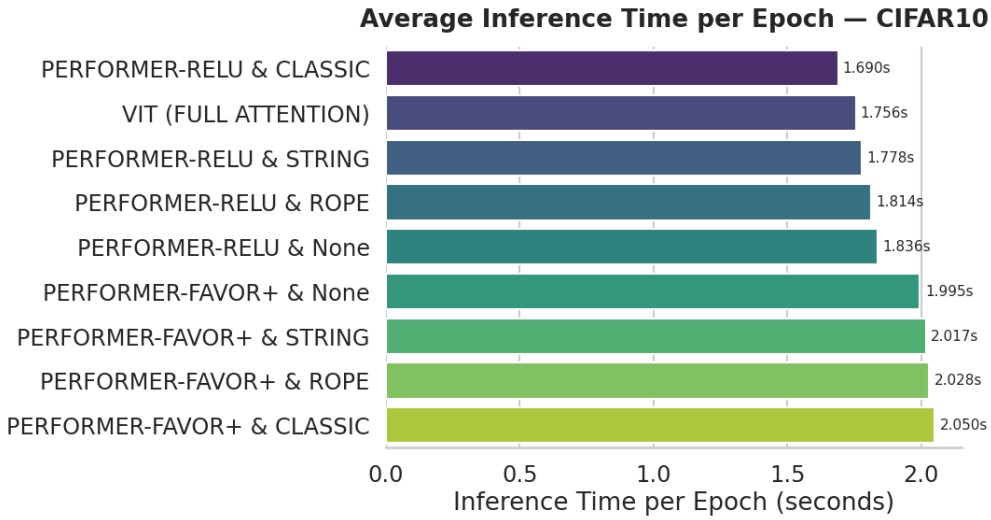

Figure 9: Average inference time per epoch on CIFAR-10. The full-attention ViT remains among the fastest models, while Performer-based models exhibit comparable inference times and RPE mechanisms introduce moderate additional overhead.

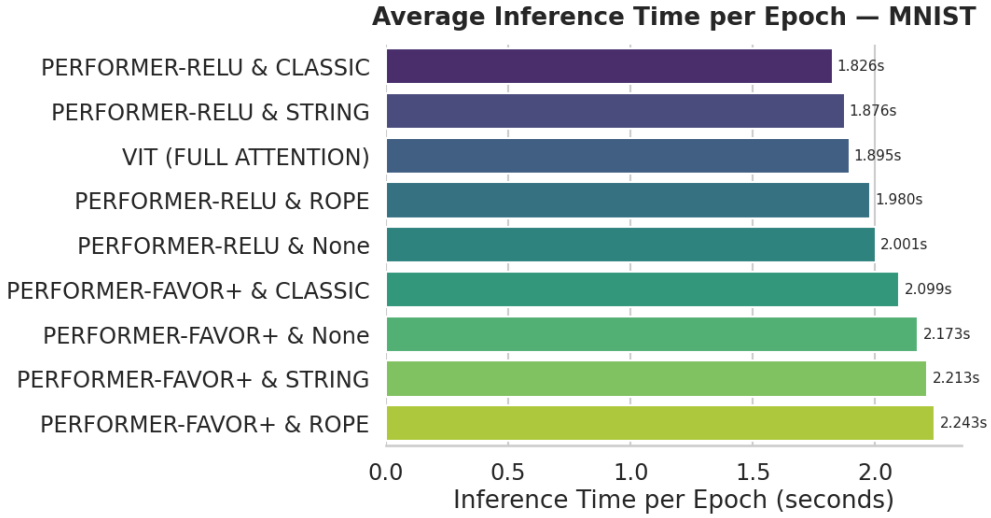

Figure 10: Average inference time per epoch on MNIST. Due to short sequences, inference-time differences across models remain limited, with the ViT and Performer variants exhibiting comparable efficiency and moderate overhead introduced by RPE mechanisms.

4 Conclusion

In this work, we investigated whether relative positional encodings (RPEs) can mitigate the accuracy degradation induced by linear attention approximations in lightweight Vision Transformers. By integrating three representative RPE mechanisms—general learned biases, rotary encodings (RoPE), and circulant-STRING—into Performer-based ViT architectures, we systematically eval-

uated their impact on accuracy, optimization stability, and computational efficiency.

Our experimental results show that Performer-based models without relative positional information often underperform both full-attention ViTs and their RPE-augmented counterparts. This gap is particularly pronounced on CIFAR-10, where linear attention approximations slow down optimization and lead to unstable training dynamics. In contrast, RoPE and circulant-STRING significantly improve convergence behavior, stabilize training, and yield substantial gains in final test accuracy for both FAVOR+ and ReLU-based Performer variants. While these RPEs do not uniformly reduce the absolute train–test gap across all settings, they consistently mitigate optimization pathologies and enable more reliable learning trajectories.

From a computational perspective, incorporating RPEs introduces additional overhead during both training and inference. However, this overhead remains moderate in absolute terms due to the short sequence length induced by image patching. Notably, the full-attention ViT remains among the fastest models at inference time in this regime, while Performer-based models exhibit inference costs that are comparable or slightly higher. Importantly, these moderate increases in inference time are systematically outweighed by the significant accuracy improvements achieved by RPE-equipped Performer models, especially on the more challenging CIFAR-10 dataset.

Overall, our results demonstrate that relative positional encodings play a critical role in restoring the representational power and training stability of linear-attention Vision Transformers. Under limited computational budgets, RoPE and circulant-STRING enable Performer-based models to substantially close the accuracy gap with full-attention ViTs while preserving favorable accuracy–efficiency trade-offs. These findings highlight the importance of positional inductive biases when deploying efficient Transformer architectures and suggest that carefully designed RPE mechanisms are essential for making linear attention competitive in practical vision applications.

[Datasets]
`MNIST`
`CIFAR-10`

References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*.

[2] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., *et al.* (2020). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. arXiv:2010.11929.

[3] Choromanski, K., Likhosherstov, V., Dohan, D., Song, X., *et al.* (2021). *Rethinking Attention with Performers*. In *Proc. of ICLR 2021*.

[4] Luo, S., Li, S., Cai, T., He, D., *et al.* (2021). *Stable, Fast and Accurate: Kernelized Attention with Relative Positional Encoding.* In *NeurIPS 2021*, pp. 22795-22807.

[5] Schenck, C., Reid, I., Jacob, M. G., Bewley, A., *et al.* (2025). *Learning the RoPEs: Better 2D and 3D Position Encodings with STRING.* In *Proc. of ICML 2025* (arXiv:2502.02562).

[6] Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). *RoFormer: Enhanced Transformer with Rotary Position Embedding. Neurocomputing*, 568:127063.

5 Code

In case the link to the GitHub repository does not work for any reason, we provide a PDF version of the code (limited to the RPE-enhanced models to avoid excessive length). The baseline model without RPE follows a very similar pipeline and differs only by the absence of the RPE modules. The code was developed on Google Colab; therefore, file paths and permissions may need to be adapted if one wishes to run it locally.
GitHub repository: `https://github.com/thomasflamand/Data_Mining_ViT_Performers`

performer_vit_RPE

December 16, 2025

[1]:
```
import math
import csv
import os
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
import torchvision
import torchvision.transforms as T

from dataclasses import dataclass
from typing import Literal

from google.colab import drive

drive.mount("/content/drive")
```

```
Mounted at /content/drive
```

[2]:
```
BASE_DIR = "/content/drive/MyDrive/Colab Notebooks/Data_Mining"

CIFAR10_PATH = f"{BASE_DIR}/cifar10"
MNIST_PATH = f"{BASE_DIR}/mnist"

TODAY = pd.Timestamp.today().strftime("%Y-%m-%d_%H-%M-%S")

RESULTS_FILE = f"{BASE_DIR}/Results/results_2_RPE_{TODAY}.csv"
```

[3]:
```
# ================================================================
# 1. Patch embedding + CLS + simple learned absolute positions
# ================================================================


class PatchEmbedding(nn.Module):
    """
    Image -> sequence of patch embeddings via a Conv2d layer.
    Args:
```

```
        img_size: size of the input image (assumed square)
        patch_size: size of each patch (assumed square)
        in_channels: number of input channels (e.g., 3 for RGB)
        embed_dim: dimension of the output embeddings
    """

    def __init__(
        self, img_size: int, patch_size: int, in_channels: int, embed_dim: int
    ):
        super().__init__()
        assert img_size % patch_size == 0, "img_size must be divisible by␣
↪patch_size"
        self.img_size = img_size
        self.patch_size = patch_size
        self.num_patches = (img_size // patch_size) ** 2
        self.proj = nn.Conv2d(
            in_channels, embed_dim, kernel_size=patch_size, stride=patch_size
        )

    def forward(self, x):
        """
        Args:
            x: Tensor of shape (B, C, H, W)
                Input batch of images.
        Returns:
            Tensor of shape (B, N, D)
                Sequence of patch embeddings, where:
                    N = number of patches
                    D = embedding dimension.
        """
        x = self.proj(x)  # (B, D, H/P, W/P)
        x = x.flatten(2)  # (B, D, N)
        x = x.transpose(1, 2)  # (B, N, D)
        return x


class ViTInputLayer(nn.Module):
    """
    PatchEmbedding + optional [CLS] token

    This module converts an input image into a sequence of patch embeddings,
    optionally prepends a trainable [CLS] token, and adds learned positional
    encodings to all tokens.
    """

    def __init__(self, img_size, patch_size, in_channels, embed_dim,␣
↪cls_token=True):
```

```
        super().__init__()

        # Convert image into sequence of patch embeddings
        self.patch_embed = PatchEmbedding(img_size, patch_size, in_channels,␣
 ↪embed_dim)

        self.num_patches = self.patch_embed.num_patches

        # Learnable CLS token (added in front of patch tokens)
        self.cls_token = (
            nn.Parameter(torch.zeros(1, 1, embed_dim)) if cls_token else None
        )

    def forward(self, x):
        B = x.size(0)
        x = self.patch_embed(x)  # (B, N, D)

        if self.cls_token is not None:
            cls = self.cls_token.expand(B, -1, -1)  # match batch size
            x = torch.cat([cls, x], dim=1)
        return x


# ================================================================
# 2. Multi-Head Attention variants
#    - Full softmax attention (baseline)
#    - Performer FAVOR+ (softmax approx with positive random features)
#    - Performer-ReLU (ReLU feature map)
# ================================================================


class MultiHeadSelfAttentionFull(nn.Module):
    def __init__(
        self,
        embed_dim: int,
        num_heads: int,
        attn_drop: float = 0.0,
        proj_drop: float = 0.0,
    ):
        super().__init__()
        assert embed_dim % num_heads == 0
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads
        self.scale = self.head_dim**-0.5

        self.qkv = nn.Linear(embed_dim, 3 * embed_dim)
        self.out_proj = nn.Linear(embed_dim, embed_dim)
```

```
        self.attn_drop = nn.Dropout(attn_drop)
        self.proj_drop = nn.Dropout(proj_drop)

    def forward(self, x):
        B, N, D = x.shape

        qkv = self.qkv(x).reshape(B, N, 3, self.num_heads, self.head_dim)
        qkv = qkv.permute(2, 0, 3, 1, 4)
        q, k, v = qkv[0], qkv[1], qkv[2]  # (B, H, N, d)

        attn_scores = torch.matmul(q, k.transpose(-2, -1)) * self.scale  #␣
↪(B,H,N,N)
        attn = F.softmax(attn_scores, dim=-1)
        attn = self.attn_drop(attn)

        out = torch.matmul(attn, v)  # (B,H,N,d)
        out = out.transpose(1, 2).reshape(B, N, D)  # (B,N,D)
        out = self.out_proj(out)
        return self.proj_drop(out)


class PerformerAttentionFavorPlus(nn.Module):
    def __init__(
        self,
        embed_dim: int,
        num_heads: int,
        nb_features: int = 64,
        attn_drop: float = 0.0,
        proj_drop: float = 0.0,
        eps: float = 1e-6,
        rpe_type: Literal["none", "rope", "classic", "string"] = "none",
        seq_len: int | None = None,
    ):
        super().__init__()

        assert embed_dim % num_heads == 0
        self.embed_dim = embed_dim
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads
        self.nb_features = nb_features
        self.eps = eps

        self.q_proj = nn.Linear(embed_dim, embed_dim)
        self.k_proj = nn.Linear(embed_dim, embed_dim)
        self.v_proj = nn.Linear(embed_dim, embed_dim)
        self.out_proj = nn.Linear(embed_dim, embed_dim)
```

```
        self.attn_drop = nn.Dropout(attn_drop)
        self.proj_drop = nn.Dropout(proj_drop)

        # Random features
        W = torch.randn(num_heads, nb_features, self.head_dim)
        self.register_buffer("W", W)

        # RPE
        self.rotary = None
        self.rpe_classic = None
        self.rpe_string = None

        if rpe_type == "rope":
            if seq_len is None:
                raise ValueError("seq_len required for RoPE")
            self.rotary = RotaryEmbedding(self.head_dim, max_seq_len=seq_len)

        elif rpe_type == "classic":
            if seq_len is None:
                raise ValueError("seq_len required for classic RPE")
            self.rpe_classic = ClassicRPEPerformer(seq_len)

        elif rpe_type == "string":
            if seq_len is None:
                raise ValueError("seq_len required for STRING RPE")
            self.rpe_string = CirculantSTRING(seq_len, self.head_dim)

    def _favor_feature_map(self, x):
        B, H, N, d_k = x.shape
        x_proj = torch.einsum("b h n d, h m d -> b h n m", x, self.W)
        sq_norm = (x**2).sum(dim=-1, keepdim=True) / 2.0
        return torch.exp(x_proj - sq_norm) / math.sqrt(self.nb_features)

    def forward(self, x):
        B, N, D = x.shape

        q = self.q_proj(x)
        k = self.k_proj(x)
        v = self.v_proj(x)

        q = q.reshape(B, N, self.num_heads, self.head_dim).permute(0, 2, 1, 3)
        k = k.reshape(B, N, self.num_heads, self.head_dim).permute(0, 2, 1, 3)
        v = v.reshape(B, N, self.num_heads, self.head_dim).permute(0, 2, 1, 3)

        # RoPE
        if self.rotary is not None:
            q, k = self.rotary(q, k)
```

```
        q_feat = self._favor_feature_map(q)
        k_feat = self._favor_feature_map(k)

        kv = torch.einsum("b h n m, b h n d -> b h m d", k_feat, v)
        k_sum = k_feat.sum(dim=2)

        denom = torch.einsum("b h n m, b h m -> b h n", q_feat, k_sum)
        denom = denom.unsqueeze(-1) + self.eps

        out = torch.einsum("b h n m, b h m d -> b h n d", q_feat, kv)
        out = out / denom  # (B,H,N,d)

        if self.rpe_classic is not None:
            weight = self.rpe_classic(N, x.device)  # (N,N)
            out = torch.einsum("b h n d, n m -> b h m d", out, weight)

        if self.rpe_string is not None:
            weight = self.rpe_string(N, x.device)  # (N,N)
            out = torch.einsum("b h n d, n m -> b h m d", out, weight)

        out = out.permute(0, 2, 1, 3).reshape(B, N, D)  # (B,N,D)

        out = self.out_proj(out)
        out = self.proj_drop(out)
        return out


class PerformerAttentionReLU(nn.Module):
    def __init__(
        self,
        embed_dim: int,
        num_heads: int,
        attn_drop: float = 0.0,
        proj_drop: float = 0.0,
        eps: float = 1e-6,
        rpe_type: Literal["none", "rope", "classic", "string"] = "none",
        seq_len: int | None = None,
    ):
        super().__init__()

        assert embed_dim % num_heads == 0
        self.embed_dim = embed_dim
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads
        self.eps = eps
```

```
        self.q_proj = nn.Linear(embed_dim, embed_dim)
        self.k_proj = nn.Linear(embed_dim, embed_dim)
        self.v_proj = nn.Linear(embed_dim, embed_dim)
        self.out_proj = nn.Linear(embed_dim, embed_dim)

        self.attn_drop = nn.Dropout(attn_drop)
        self.proj_drop = nn.Dropout(proj_drop)

        # RPE
        self.rotary = None
        self.rpe_classic = None
        self.rpe_string = None

        if rpe_type == "rope":
            if seq_len is None:
                raise ValueError("seq_len required for RoPE")
            self.rotary = RotaryEmbedding(self.head_dim, max_seq_len=seq_len)

        elif rpe_type == "classic":
            if seq_len is None:
                raise ValueError("seq_len required for classic RPE")
            self.rpe_classic = ClassicRPEPerformer(seq_len)

        elif rpe_type == "string":
            if seq_len is None:
                raise ValueError("seq_len required for STRING RPE")
            self.rpe_string = CirculantSTRING(seq_len, self.head_dim)

    def _relu_feature_map(self, x):
        return F.relu(x)

    def forward(self, x):
        B, N, D = x.shape

        q = self.q_proj(x)
        k = self.k_proj(x)
        v = self.v_proj(x)

        q = q.reshape(B, N, self.num_heads, self.head_dim).permute(0, 2, 1, 3)
        k = k.reshape(B, N, self.num_heads, self.head_dim).permute(0, 2, 1, 3)
        v = v.reshape(B, N, self.num_heads, self.head_dim).permute(0, 2, 1, 3)

        # RoPE
        if self.rotary is not None:
            q, k = self.rotary(q, k)

        q_feat = self._relu_feature_map(q)
```

```
        k_feat = self._relu_feature_map(k)

        kv = torch.einsum("b h n d, b h n e -> b h d e", k_feat, v)
        k_sum = k_feat.sum(dim=2)

        denom = torch.einsum("b h n d, b h d -> b h n", q_feat, k_sum)
        denom = denom.unsqueeze(-1) + self.eps

        out = torch.einsum("b h n d, b h d e -> b h n e", q_feat, kv)
        out = out / denom  # (B,H,N,d)

        # Classic RPE multiplicative
        if self.rpe_classic is not None:
            weight = self.rpe_classic(N, x.device)  # (N,N)
            out = torch.einsum("b h n e, n m -> b h m e", out, weight)

        # STRING RPE
        if self.rpe_string is not None:
            weight = self.rpe_string(N, x.device)  # (N,N)
            out = torch.einsum("b h n e, n m -> b h m e", out, weight)

        out = out.permute(0, 2, 1, 3).reshape(B, N, D)

        out = self.out_proj(out)
        out = self.proj_drop(out)
        return out


# ================================================================
# 3. Transformer block + ViT backbone
# ================================================================


class MLPBlock(nn.Module):
    """
    Standard Transformer feedforward block:
    Linear -> GELU -> Linear (with optional dropout).

    Position-wise feedforward network that expands and compresses each token
    to increase the transformer's expressive capacity.
    """

    def __init__(self, embed_dim: int, mlp_ratio: float = 4.0, drop: float = 0.
 ↪0):
        super().__init__()
        hidden_dim = int(embed_dim * mlp_ratio)
```

```
        self.fc1 = nn.Linear(embed_dim, hidden_dim)  # expansion layer
        self.act = nn.GELU()  # non-linearity
        self.fc2 = nn.Linear(hidden_dim, embed_dim)  # projection back to D
        self.drop = nn.Dropout(drop)

    def forward(self, x):
        x = self.fc1(x)  # (B, N, hidden_dim)
        x = self.act(x)  # GELU activation
        x = self.drop(x)  # dropout after activation
        x = self.fc2(x)  # back to (B, N, D)
        x = self.drop(x)  # optional dropout on output
        return x


class TransformerEncoderBlock(nn.Module):
    """
    Pre-LN Transformer block with selectable attention type
    """

    def __init__(
        self,
        embed_dim: int,
        num_heads: int,
        mlp_ratio: float = 4.0,
        attn_type: Literal["full", "favor+", "relu"] = "full",
        nb_features: int = 64,
        drop: float = 0.0,
        attn_drop: float = 0.0,
        rpe_type: Literal["none", "rope", "classic", "string"] = "none",
        seq_len: int | None = None,
    ):
        super().__init__()

        self.norm1 = nn.LayerNorm(embed_dim)
        self.norm2 = nn.LayerNorm(embed_dim)

        if attn_type == "full":
            self.attn = MultiHeadSelfAttentionFull(
                embed_dim,
                num_heads,
                attn_drop,
                drop,
            )

        elif attn_type == "favor+":
            self.attn = PerformerAttentionFavorPlus(
                embed_dim,
```

```
                num_heads,
                nb_features,
                attn_drop,
                drop,
                rpe_type=rpe_type,
                seq_len=seq_len,
            )
        elif attn_type == "relu":
            self.attn = PerformerAttentionReLU(
                embed_dim,
                num_heads,
                attn_drop=attn_drop,
                proj_drop=drop,
                rpe_type=rpe_type,
                seq_len=seq_len,
            )

        self.mlp = MLPBlock(embed_dim, mlp_ratio, drop)

    def forward(self, x):
        x = x + self.attn(self.norm1(x))
        x = x + self.mlp(self.norm2(x))
        return x


class ViTClassifier(nn.Module):
    """
    ViT / Performer-ViT for MNIST / CIFAR-10
    """

    def __init__(
        self,
        img_size: int,
        patch_size: int,
        in_channels: int,
        num_classes: int,
        embed_dim: int = 64,
        depth: int = 4,
        num_heads: int = 4,
        mlp_ratio: float = 4.0,
        attn_type: Literal["full", "favor+", "relu"] = "full",
        nb_features: int = 64,
        drop: float = 0.0,
        attn_drop: float = 0.0,
        rpe_type: Literal["none", "rope", "classic", "string"] = "none",
    ):
        super().__init__()
```

```
        self.input_layer = ViTInputLayer(
            img_size, patch_size, in_channels, embed_dim, cls_token=True
        )

        seq_len = 1 + (img_size // patch_size) ** 2

        self.blocks = nn.ModuleList(
            [
                TransformerEncoderBlock(
                    embed_dim=embed_dim,
                    num_heads=num_heads,
                    mlp_ratio=mlp_ratio,
                    attn_type=attn_type,
                    nb_features=nb_features,
                    drop=drop,
                    attn_drop=attn_drop,
                    rpe_type=rpe_type,
                    seq_len=seq_len,
                )
                for _ in range(depth)
            ]
        )

        self.norm = nn.LayerNorm(embed_dim)
        self.head = nn.Linear(embed_dim, num_classes)

    def forward(self, x):
        x = self.input_layer(x)
        for blk in self.blocks:
            x = blk(x)
        x = self.norm(x)
        cls_token = x[:, 0]
        return self.head(cls_token)
```

[ ]:
```
# ----------------------------------------------------------
# RPE MODULES
# ----------------------------------------------------------


class ClassicRPEPerformer(nn.Module):
    """
    Classic multiplicative RPE for Performer (Luo et al., 2021)
    Applied as a kernel weight: w[i,j]
    """

    def __init__(self, seq_len):
```

```
        super().__init__()
        self.seq_len = seq_len
        self.rpe = nn.Parameter(torch.zeros(seq_len))  # 1D kernel

    def forward(self, N, device):
        idx = torch.arange(N, device=device)

        # directional: (j - i) mod N
        rel = (idx[None, :] - idx[:, None]) % N

        # kernel weighting
        weight = torch.exp(self.rpe[rel])  # (N,N)
        return weight


class CirculantSTRING(nn.Module):
    """
    Circulant-STRING RPE : génère une matrice (N,N) de pondération␣
 ↪(directionnelle).
    Utilisée uniquement dans les Performers.
    """

    def __init__(self, seq_len: int, head_dim: int):
        super().__init__()
        self.seq_len = seq_len
        self.kernel = nn.Parameter(torch.zeros(seq_len))
        self.scale = 1.0 / math.sqrt(head_dim)

    def forward(self, N: int, device: torch.device):
        if N != self.seq_len:
            raise ValueError(
                f"CirculantSTRING requires fixed seq_len={self.seq_len}, got␣
 ↪N={N}"
            )

        kernel_fft = torch.fft.rfft(self.kernel, n=N)  # (N_fft,)

        eye = torch.eye(N, device=device)  # (N,N)
        eye_fft = torch.fft.rfft(eye, n=N, dim=-1)  # (N, N_fft)

        out = torch.fft.irfft(eye_fft * kernel_fft[None, :], n=N)  # (N,N)
        return self.scale * out  # (N,N)


class RotaryEmbedding(nn.Module):
    def __init__(self, head_dim: int, max_seq_len: int = 512):
        super().__init__()
```

```
        if head_dim % 2 != 0:
            raise ValueError("RoPE nécessite un head_dim pair")

        self.head_dim = head_dim
        self.max_seq_len = max_seq_len

        inv_freq = 1.0 / (10000 ** (torch.arange(0, head_dim, 2).float() /␣
↪head_dim))
        self.register_buffer("inv_freq", inv_freq)

    def _build_sin_cos(self, seq_len: int, device: torch.device):
        # positions (N,)
        pos = torch.arange(seq_len, dtype=torch.float32, device=device)  # (N,)

        # freqs = (N, head_dim/2)
        freqs = torch.einsum("n,d->nd", pos, self.inv_freq)

        # sin/cos = (N, head_dim/2)
        sin = torch.sin(freqs)
        cos = torch.cos(freqs)

        sin = torch.stack([sin, sin], dim=-1).reshape(seq_len, self.head_dim)
        cos = torch.stack([cos, cos], dim=-1).reshape(seq_len, self.head_dim)

        # reshape (1,1,N,head_dim)
        return (
            sin.unsqueeze(0).unsqueeze(0),
            cos.unsqueeze(0).unsqueeze(0),
        )

    @staticmethod
    def _rotate_half(x: torch.Tensor) -> torch.Tensor:
        d = x.shape[-1]
        x1 = x[..., : d // 2]
        x2 = x[..., d // 2 :]
        return torch.cat([-x2, x1], dim=-1)

    def forward(self, q, k):
        B, H, N, d = q.shape
        if N > self.max_seq_len:
            raise ValueError(f"seq_len {N} > max_seq_len {self.max_seq_len}")

        sin, cos = self._build_sin_cos(N, q.device)

        q_rot = (q * cos) + (self._rotate_half(q) * sin)
        k_rot = (k * cos) + (self._rotate_half(k) * sin)
        return q_rot, k_rot
```

```python
# ================================================================
# 4. Helper functions to build model from config
# ================================================================
@dataclass
class ViTConfig:
    img_size: int = 32
    patch_size: int = 4
    in_channels: int = 3
    num_classes: int = 10
    embed_dim: int = 64
    depth: int = 4
    num_heads: int = 4
    mlp_ratio: float = 4.0
    attn_type: str = "favor+"
    nb_features: int = 64
    drop: float = 0.0
    attn_drop: float = 0.0
    rpe_type: Literal["none", "rope", "classic", "string"] = "none"


def build_model(cfg: ViTConfig) -> nn.Module:
    return ViTClassifier(
        img_size=cfg.img_size,
        patch_size=cfg.patch_size,
        in_channels=cfg.in_channels,
        num_classes=cfg.num_classes,
        embed_dim=cfg.embed_dim,
        depth=cfg.depth,
        num_heads=cfg.num_heads,
        mlp_ratio=cfg.mlp_ratio,
        attn_type=cfg.attn_type,
        nb_features=cfg.nb_features,
        drop=cfg.drop,
        attn_drop=cfg.attn_drop,
        rpe_type=cfg.rpe_type,
    )


# ================================================================
# 5. Training loop for MNIST / CIFAR-10 (simple baseline)
# ================================================================
import csv
import time
```

```python
def get_mnist_loaders(batch_size=128):
    # Basic preprocessing: resize to 32×32 then convert to tensor
    transform = T.Compose(
        [
            T.Resize(32),
            T.ToTensor(),
        ]
    )

    # Load MNIST train/test splits
    train_set = torchvision.datasets.MNIST(
        root=MNIST_PATH, train=True, download=True, transform=transform
    )
    test_set = torchvision.datasets.MNIST(
        root=MNIST_PATH, train=False, download=True, transform=transform
    )

    # Wrap into DataLoader objects
    train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True)
    test_loader = DataLoader(test_set, batch_size=batch_size, shuffle=False)
    return train_loader, test_loader


def get_cifar10_loaders(batch_size=128):
    # Data augmentation for training
    transform_train = T.Compose(
        [
            T.RandomHorizontalFlip(),
            T.ToTensor(),
        ]
    )

    # Standard preprocessing for test
    transform_test = T.Compose([T.ToTensor()])

    # Load CIFAR-10 train/test splits
    train_set = torchvision.datasets.CIFAR10(
        root=CIFAR10_PATH, train=True, download=True, transform=transform_train
    )
    test_set = torchvision.datasets.CIFAR10(
        root=CIFAR10_PATH, train=False, download=True, transform=transform_test
    )

    # Wrap into DataLoaders
    train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True)
    test_loader = DataLoader(test_set, batch_size=batch_size, shuffle=False)
    return train_loader, test_loader
```

```python
def train_one_epoch(model, loader, optimizer, device):
    model.train()
    total_loss = 0.0
    total_correct = 0
    total_samples = 0
    start_time = time.time()

    for x, y in loader:
        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()
        logits = model(x)  # forward pass
        loss = F.cross_entropy(logits, y)  # classification loss
        loss.backward()  # backprop
        optimizer.step()  # update weights

        # Accumulate stats
        total_loss += loss.item() * x.size(0)
        preds = logits.argmax(dim=-1)
        total_correct += (preds == y).sum().item()
        total_samples += x.size(0)

    end_time = time.time()
    elapsed_time = end_time - start_time

    # Return average loss, accuracy, and elapsed time
    return total_loss / total_samples, total_correct / total_samples,␣
 ↪elapsed_time


def evaluate(model, loader, device):
    model.eval()
    total_loss = 0.0
    total_correct = 0
    total_samples = 0
    start_time = time.time()

    with torch.no_grad():
        for x, y in loader:
            x = x.to(device)
            y = y.to(device)

            logits = model(x)
            loss = F.cross_entropy(logits, y)
```

```
            total_loss += loss.item() * x.size(0)
            preds = logits.argmax(dim=-1)
            total_correct += (preds == y).sum().item()
            total_samples += x.size(0)

    end_time = time.time()
    elapsed_time = end_time - start_time

    return total_loss / total_samples, total_correct / total_samples,␣
 ↪elapsed_time


# ================================================================
# CSV logging helper (appends without overwriting existing data)
# ================================================================


def log_results(filepath, row_dict):
    file_exists = os.path.isfile(filepath)
    with open(filepath, "a", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=row_dict.keys())
        if not file_exists:
            writer.writeheader()
        writer.writerow(row_dict)


# ================================================================
# RUN EXAMPLE
# ================================================================


def run_training_RPE(
    dataset: Literal["mnist", "cifar10"] = "mnist",
    attn_type: Literal["full", "favor+", "relu"] = "full",
    rpe_type: Literal["none", "rope", "classic", "string"] = "none",
    epochs: int = 5,
    lr: float = 1e-3,
    batch_size: int = 128,
):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")

    # 1) Dataset selection
    if dataset == "mnist":
        train_loader, test_loader = get_mnist_loaders(batch_size)
        cfg = ViTConfig(
```

```
            img_size=32,
            patch_size=4,
            in_channels=1,
            num_classes=10,
            embed_dim=64,
            depth=4,
            num_heads=4,
            mlp_ratio=4.0,
            attn_type=attn_type,
            nb_features=64,
            rpe_type=rpe_type,
        )

    elif dataset == "cifar10":
        train_loader, test_loader = get_cifar10_loaders(batch_size)
        cfg = ViTConfig(
            img_size=32,
            patch_size=4,
            in_channels=3,
            num_classes=10,
            embed_dim=64,
            depth=4,
            num_heads=4,
            mlp_ratio=4.0,
            attn_type=attn_type,
            nb_features=64,
            rpe_type=rpe_type,
        )

    # 2) Build model
    model = build_model(cfg).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    # 3) Training loop
    for epoch in range(1, epochs + 1):
        train_loss, train_acc, train_time = train_one_epoch(
            model, train_loader, optimizer, device
        )
        test_loss, test_acc, eval_time = evaluate(model, test_loader, device)

        print(
            f"[{dataset}][attn={attn_type}][rpe={rpe_type}] "
            f"Epoch {epoch:02d}: "
            f"train loss={train_loss:.4f}, train acc={train_acc:.4f}, "
            f"test loss={test_loss:.4f}, test acc={test_acc:.4f}"
        )
```

```
        # Log to CSV
        log_results(
            RESULTS_FILE,
            {
                "dataset": dataset,
                "attn_type": attn_type,
                "rpe_type": rpe_type,
                "epochs_total": epochs,
                "epoch": epoch,
                "lr": lr,
                "batch_size": batch_size,
                "train_loss": train_loss,
                "train_acc": train_acc,
                "train_time_sec": train_time,
                "test_loss": test_loss,
                "test_acc": test_acc,
                "eval_time_sec": eval_time,
            },
        )


# datasets = ["mnist", "cifar10"]
datasets = ["cifar10"]
attn_types = ["favor+", "relu"]
rpe_types_full = ["none"]
rpe_types_performer = ["none", "rope", "classic", "string"]

EPOCHS_MNIST = 5
EPOCHS_CIFAR = 20

for dataset in datasets:
    for attn in attn_types:
        if attn == "full":
            to_run = rpe_types_full
        else:
            to_run = rpe_types_performer

        for rpe in to_run:
            epochs = EPOCHS_MNIST if dataset == "mnist" else EPOCHS_CIFAR

            print("\n==========================================")
            print(
                f" RUN: dataset={dataset} | attn={attn} | rpe={rpe} |␣
 ↪epochs={epochs}"
            )
            print("==========================================\n")
```

```
            run_training_RPE(
                dataset=dataset,
                attn_type=attn,
                rpe_type=rpe,
                epochs=epochs,
                lr=1e-3,
                batch_size=128,
            )
```

[ ]:
```
!ls -l "/content"
```

[ ]:
```
!ls -l "/content/drive/MyDrive/Colab Notebooks"
```

1 Special case used to maximize accuracy (for explanatory purposes)

[5]:
```
from torch.optim.lr_scheduler import CosineAnnealingLR
RESULTS_FILE_MAX = f"{BASE_DIR}/Results/results_2_RPE_MAX_{TODAY}.csv"

# ================================================================
# 1. Patch embedding + CLS + simple learned absolute positions
# ================================================================

class PatchEmbedding(nn.Module):

    def __init__(
        self, img_size: int, patch_size: int, in_channels: int, embed_dim: int
    ):
        super().__init__()
        assert img_size % patch_size == 0,
        self.img_size = img_size
        self.patch_size = patch_size
        self.num_patches = (img_size // patch_size) ** 2
        self.proj = nn.Conv2d(
            in_channels, embed_dim, kernel_size=patch_size, stride=patch_size
        )

    def forward(self, x):
        x = self.proj(x)
        x = x.flatten(2)
        x = x.transpose(1, 2)
        return x


class ViTInputLayer(nn.Module):
```

```
    def __init__(self, img_size, patch_size, in_channels, embed_dim,␣
 ↪cls_token=True):
        super().__init__()

        self.patch_embed = PatchEmbedding(img_size, patch_size, in_channels,␣
 ↪embed_dim)

        self.num_patches = self.patch_embed.num_patches

        self.cls_token = (
            nn.Parameter(torch.zeros(1, 1, embed_dim)) if cls_token else None
        )

    def forward(self, x):
        B = x.size(0)
        x = self.patch_embed(x)

        if self.cls_token is not None:
            cls = self.cls_token.expand(B, -1, -1)
            x = torch.cat([cls, x], dim=1)
        return x

# ================================================================
# 2. Multi-Head Attention variants
#    - Full softmax attention (baseline)
#    - Performer FAVOR+ (softmax approx with positive random features)
#    - Performer-ReLU (ReLU feature map)
# ================================================================

class MultiHeadSelfAttentionFull(nn.Module):
    def __init__(
        self,
        embed_dim: int,
        num_heads: int,
        attn_drop: float = 0.0,
        proj_drop: float = 0.0,
    ):
        super().__init__()
        assert embed_dim % num_heads == 0
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads
        self.scale = self.head_dim**-0.5

        self.qkv = nn.Linear(embed_dim, 3 * embed_dim)
        self.out_proj = nn.Linear(embed_dim, embed_dim)
        self.attn_drop = nn.Dropout(attn_drop)
        self.proj_drop = nn.Dropout(proj_drop)
```

```
    def forward(self, x):
        B, N, D = x.shape

        qkv = self.qkv(x).reshape(B, N, 3, self.num_heads, self.head_dim)
        qkv = qkv.permute(2, 0, 3, 1, 4)
        q, k, v = qkv[0], qkv[1], qkv[2]

        attn_scores = torch.matmul(q, k.transpose(-2, -1)) * self.scale
        attn = F.softmax(attn_scores, dim=-1)
        attn = self.attn_drop(attn)

        out = torch.matmul(attn, v)
        out = out.transpose(1, 2).reshape(B, N, D)
        out = self.out_proj(out)
        return self.proj_drop(out)

class PerformerAttentionFavorPlus(nn.Module):
    def __init__(
        self,
        embed_dim: int,
        num_heads: int,
        nb_features: int = 64,
        attn_drop: float = 0.0,
        proj_drop: float = 0.0,
        eps: float = 1e-6,
        rpe_type: Literal["none", "rope", "classic", "string"] = "none",
        seq_len: int | None = None,
    ):
        super().__init__()

        assert embed_dim % num_heads == 0
        self.embed_dim = embed_dim
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads
        self.nb_features = nb_features
        self.eps = eps

        self.q_proj = nn.Linear(embed_dim, embed_dim)
        self.k_proj = nn.Linear(embed_dim, embed_dim)
        self.v_proj = nn.Linear(embed_dim, embed_dim)
        self.out_proj = nn.Linear(embed_dim, embed_dim)

        self.attn_drop = nn.Dropout(attn_drop)
        self.proj_drop = nn.Dropout(proj_drop)

        W = torch.randn(num_heads, nb_features, self.head_dim)
```

```
        self.register_buffer("W", W)

        self.rotary = None
        self.rpe_classic = None
        self.rpe_string = None

        if rpe_type == "rope":
            if seq_len is None:
                raise ValueError("seq_len required for RoPE")
            self.rotary = RotaryEmbedding(self.head_dim, max_seq_len=seq_len)

        elif rpe_type == "classic":
            if seq_len is None:
                raise ValueError("seq_len required for classic RPE")
            self.rpe_classic = ClassicRPEPerformer(seq_len)

        elif rpe_type == "string":
            if seq_len is None:
                raise ValueError("seq_len required for STRING RPE")
            self.rpe_string = CirculantSTRING(seq_len, self.head_dim)

    def _favor_feature_map(self, x):
        B, H, N, d_k = x.shape
        x_proj = torch.einsum("b h n d, h m d -> b h n m", x, self.W)
        sq_norm = (x**2).sum(dim=-1, keepdim=True) / 2.0
        return torch.exp(x_proj - sq_norm) / math.sqrt(self.nb_features)

    def forward(self, x):
        B, N, D = x.shape

        q = self.q_proj(x)
        k = self.k_proj(x)
        v = self.v_proj(x)

        q = q.reshape(B, N, self.num_heads, self.head_dim).permute(0, 2, 1, 3)
        k = k.reshape(B, N, self.num_heads, self.head_dim).permute(0, 2, 1, 3)
        v = v.reshape(B, N, self.num_heads, self.head_dim).permute(0, 2, 1, 3)

        if self.rotary is not None:
            q, k = self.rotary(q, k)

        q_feat = self._favor_feature_map(q)
        k_feat = self._favor_feature_map(k)

        kv = torch.einsum("b h n m, b h n d -> b h m d", k_feat, v)
        k_sum = k_feat.sum(dim=2)
```

```
        denom = torch.einsum("b h n m, b h m -> b h n", q_feat, k_sum)
        denom = denom.unsqueeze(-1) + self.eps

        out = torch.einsum("b h n m, b h m d -> b h n d", q_feat, kv)
        out = out / denom

        if self.rpe_classic is not None:
            weight = self.rpe_classic(N, x.device)
            out = torch.einsum("b h n d, n m -> b h m d", out, weight)

        if self.rpe_string is not None:
            weight = self.rpe_string(N, x.device)
            out = torch.einsum("b h n d, n m -> b h m d", out, weight)

        out = out.permute(0, 2, 1, 3).reshape(B, N, D)

        out = self.out_proj(out)
        out = self.proj_drop(out)
        return out


# ============================================================
# 3. Transformer block + ViT backbone
# ============================================================

class MLPBlock(nn.Module):
    def __init__(self, embed_dim: int, mlp_ratio: float = 4.0, drop: float = 0.
 ↪0):
        super().__init__()
        hidden_dim = int(embed_dim * mlp_ratio)

        self.fc1 = nn.Linear(embed_dim, hidden_dim)
        self.act = nn.GELU()
        self.fc2 = nn.Linear(hidden_dim, embed_dim)
        self.drop = nn.Dropout(drop)

    def forward(self, x):
        x = self.fc1(x)
        x = self.act(x)
        x = self.drop(x)
        x = self.fc2(x)
        x = self.drop(x)
        return x

class TransformerEncoderBlock(nn.Module):
  def __init__(
      self,
```

```
    embed_dim: int,
    num_heads: int,
    mlp_ratio: float = 4.0,
    attn_type: Literal["full", "favor+", "relu"] = "full",
    nb_features: int = 64,
    drop: float = 0.0,
    attn_drop: float = 0.0,
    rpe_type: Literal["none", "rope", "classic", "string"] = "none",
    seq_len: int | None = None,
):
    super().__init__()

    self.norm1 = nn.LayerNorm(embed_dim)
    self.norm2 = nn.LayerNorm(embed_dim)

    if attn_type == "full":
        self.attn = MultiHeadSelfAttentionFull(
            embed_dim,
            num_heads,
            attn_drop,
            drop,
        )

    elif attn_type == "favor+":
        self.attn = PerformerAttentionFavorPlus(
            embed_dim,
            num_heads,
            nb_features,
            attn_drop,
            drop,
            rpe_type=rpe_type,
            seq_len=seq_len,
        )
    elif attn_type == "relu":
        self.attn = PerformerAttentionReLU(
            embed_dim,
            num_heads,
            attn_drop=attn_drop,
            proj_drop=drop,
            rpe_type=rpe_type,
            seq_len=seq_len,
        )

    self.mlp = MLPBlock(embed_dim, mlp_ratio, drop)

def forward(self, x):
    x = x + self.attn(self.norm1(x))
```

```
      x = x + self.mlp(self.norm2(x))
      return x

class ViTClassifier(nn.Module):
    def __init__(
        self,
        img_size: int,
        patch_size: int,
        in_channels: int,
        num_classes: int,
        embed_dim: int = 64,
        depth: int = 4,
        num_heads: int = 4,
        mlp_ratio: float = 4.0,
        attn_type: Literal["full", "favor+", "relu"] = "full",
        nb_features: int = 64,
        drop: float = 0.0,
        attn_drop: float = 0.0,
        rpe_type: Literal["none", "rope", "classic", "string"] = "none",
    ):
        super().__init__()

        self.input_layer = ViTInputLayer(
            img_size, patch_size, in_channels, embed_dim, cls_token=True
        )

        seq_len = 1 + (img_size // patch_size) ** 2

        self.blocks = nn.ModuleList(
            [
                TransformerEncoderBlock(
                    embed_dim=embed_dim,
                    num_heads=num_heads,
                    mlp_ratio=mlp_ratio,
                    attn_type=attn_type,
                    nb_features=nb_features,
                    drop=drop,
                    attn_drop=attn_drop,
                    rpe_type=rpe_type,
                    seq_len=seq_len,
                )
                for _ in range(depth)
            ]
        )

        self.norm = nn.LayerNorm(embed_dim)
        self.head = nn.Linear(embed_dim, num_classes)
```

```
    def forward(self, x):
        x = self.input_layer(x)
        for blk in self.blocks:
            x = blk(x)
        x = self.norm(x)
        cls_token = x[:, 0]
        return self.head(cls_token)
```

[ ]:
```
# ----------------------------------------------------------
# RPE MODULES
# ----------------------------------------------------------


class RotaryEmbedding(nn.Module):
    def __init__(self, head_dim: int, max_seq_len: int = 512):
        super().__init__()
        if head_dim % 2 != 0:
            raise ValueError("RoPE nécessite un head_dim pair")

        self.head_dim = head_dim
        self.max_seq_len = max_seq_len

        inv_freq = 1.0 / (10000 ** (torch.arange(0, head_dim, 2).float() /␣
 ↪head_dim))
        self.register_buffer("inv_freq", inv_freq)

    def _build_sin_cos(self, seq_len: int, device: torch.device):
        pos = torch.arange(seq_len, dtype=torch.float32, device=device)  # (N,)
        freqs = torch.einsum("n,d->nd", pos, self.inv_freq)

        sin = torch.sin(freqs)
        cos = torch.cos(freqs)

        sin = torch.stack([sin, sin], dim=-1).reshape(seq_len, self.head_dim)
        cos = torch.stack([cos, cos], dim=-1).reshape(seq_len, self.head_dim)

        return (
            sin.unsqueeze(0).unsqueeze(0),
            cos.unsqueeze(0).unsqueeze(0),
        )

    @staticmethod
    def _rotate_half(x: torch.Tensor) -> torch.Tensor:
        d = x.shape[-1]
        x1 = x[..., : d // 2]
        x2 = x[..., d // 2 :]
```

```
        return torch.cat([-x2, x1], dim=-1)

    def forward(self, q, k):
        B, H, N, d = q.shape
        if N > self.max_seq_len:
            raise ValueError(f"seq_len {N} > max_seq_len {self.max_seq_len}")

        sin, cos = self._build_sin_cos(N, q.device)

        q_rot = (q * cos) + (self._rotate_half(q) * sin)
        k_rot = (k * cos) + (self._rotate_half(k) * sin)
        return q_rot, k_rot


# ================================================================
# 4. Helper functions to build model from config
# ================================================================
@dataclass
class ViTConfig:
    img_size: int = 32
    patch_size: int = 4
    in_channels: int = 3
    num_classes: int = 10
    embed_dim: int = 64
    depth: int = 4
    num_heads: int = 4
    mlp_ratio: float = 4.0
    attn_type: str = "favor+"
    nb_features: int = 64
    drop: float = 0.0
    attn_drop: float = 0.0
    rpe_type: Literal["rope"] = "rope"
    drop: float = 0.1
    attn_drop: float = 0.1


def build_model(cfg: ViTConfig) -> nn.Module:
    return ViTClassifier(
        img_size=cfg.img_size,
        patch_size=cfg.patch_size,
        in_channels=cfg.in_channels,
        num_classes=cfg.num_classes,
        embed_dim=cfg.embed_dim,
        depth=cfg.depth,
        num_heads=cfg.num_heads,
        mlp_ratio=cfg.mlp_ratio,
        attn_type=cfg.attn_type,
```

```
        nb_features=cfg.nb_features,
        drop=cfg.drop,
        attn_drop=cfg.attn_drop,
        rpe_type=cfg.rpe_type,
    )


# ================================================================
# 5. Training loop for MNIST / CIFAR-10 (simple baseline)
# ================================================================
import csv
import time


def get_mnist_loaders(batch_size=128):
    transform = T.Compose(
        [
            T.Resize(32),
            T.ToTensor(),
        ]
    )

    train_set = torchvision.datasets.MNIST(
        root=MNIST_PATH, train=True, download=True, transform=transform
    )
    test_set = torchvision.datasets.MNIST(
        root=MNIST_PATH, train=False, download=True, transform=transform
    )

    train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True)
    test_loader = DataLoader(test_set, batch_size=batch_size, shuffle=False)
    return train_loader, test_loader


def get_cifar10_loaders(batch_size=128):
    transform_train = T.Compose(
        [
            T.RandomHorizontalFlip(),
            T.ToTensor(),
        ]
    )

    transform_test = T.Compose([T.ToTensor()])

    train_set = torchvision.datasets.CIFAR10(
        root=CIFAR10_PATH, train=True, download=True, transform=transform_train
    )
```

```
    test_set = torchvision.datasets.CIFAR10(
        root=CIFAR10_PATH, train=False, download=True, transform=transform_test
    )

    train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True)
    test_loader = DataLoader(test_set, batch_size=batch_size, shuffle=False)
    return train_loader, test_loader


def train_one_epoch(model, loader, optimizer, device):
    model.train()
    total_loss = 0.0
    total_correct = 0
    total_samples = 0
    start_time = time.time()

    for x, y in loader:
        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()
        logits = model(x)
        loss = F.cross_entropy(logits, y)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * x.size(0)
        preds = logits.argmax(dim=-1)
        total_correct += (preds == y).sum().item()
        total_samples += x.size(0)

    end_time = time.time()
    elapsed_time = end_time - start_time

    return total_loss / total_samples, total_correct / total_samples,␣
 ↪elapsed_time


def evaluate(model, loader, device):
    model.eval()
    total_loss = 0.0
    total_correct = 0
    total_samples = 0
    start_time = time.time()

    with torch.no_grad():
        for x, y in loader:
```

```
            x = x.to(device)
            y = y.to(device)

            logits = model(x)
            loss = F.cross_entropy(logits, y)

            total_loss += loss.item() * x.size(0)
            preds = logits.argmax(dim=-1)
            total_correct += (preds == y).sum().item()
            total_samples += x.size(0)

    end_time = time.time()
    elapsed_time = end_time - start_time

    return total_loss / total_samples, total_correct / total_samples,␣
 ↪elapsed_time


# ================================================================
# CSV logging helper (appends without overwriting existing data)
# ================================================================


def log_results(filepath, row_dict):
    file_exists = os.path.isfile(filepath)
    with open(filepath, "a", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=row_dict.keys())
        if not file_exists:
            writer.writeheader()
        writer.writerow(row_dict)


# ================================================================
# RUN EXAMPLE
# ================================================================


def run_training_RPE_maxperf(dataset="cifar10"):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")

    epochs = 60 if dataset == "cifar10" else 10

    if dataset == "cifar10":
        train_loader, test_loader = get_cifar10_loaders(128)
        cfg = ViTConfig(
            img_size=32,
```

```
            patch_size=4,
            in_channels=3,
            num_classes=10,
            embed_dim=64,
            depth=4,
            num_heads=4,
            mlp_ratio=4.0,
            attn_type="favor+",
            nb_features=64,
            rpe_type="rope",
            drop=0.1,
            attn_drop=0.1,
        )
    else:
        train_loader, test_loader = get_mnist_loaders(128)
        cfg = ViTConfig(
            img_size=32,
            patch_size=4,
            in_channels=1,
            num_classes=10,
            embed_dim=64,
            depth=4,
            num_heads=4,
            mlp_ratio=4.0,
            attn_type="favor+",
            nb_features=64,
            rpe_type="rope",
            drop=0.1,
            attn_drop=0.1,
        )

    model = build_model(cfg).to(device)

    optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.
↪05)

    scheduler = CosineAnnealingLR(optimizer, T_max=epochs)
    warmup_epochs = 5

    for epoch in range(1, epochs + 1):
        if epoch <= warmup_epochs:
            lr_scale = epoch / warmup_epochs
            for pg in optimizer.param_groups:
                pg["lr"] = lr_scale * 3e-4

        train_loss, train_acc, _ = train_one_epoch(
            model, train_loader, optimizer, device
```

```
        )
        test_loss, test_acc, _ = evaluate(model, test_loader, device)

        scheduler.step()

        print(
            f"[{dataset}] Epoch {epoch:03d} | "
            f"train acc={train_acc:.4f} | test acc={test_acc:.4f}"
        )

        log_results(
            RESULTS_FILE_MAX,
            {
                "dataset": dataset,
                "attn_type": attn_type,
                "rpe_type": rpe_type,
                "epochs_total": epochs,
                "epoch": epoch,
                "lr": lr,
                "batch_size": batch_size,
                "train_loss": train_loss,
                "train_acc": train_acc,
                "train_time_sec": train_time,
                "test_loss": test_loss,
                "test_acc": test_acc,
                "eval_time_sec": eval_time,
            },
        )


datasets = ["cifar10"]
attn_types = ["favor+"]
rpe_types_full = ["none"]
rpe_types_performer = ["rope"]

EPOCHS_MNIST = 10
EPOCHS_CIFAR = 60

for dataset in datasets:
    for attn in attn_types:
        to_run = rpe_types_performer
        for rpe in to_run:
            epochs = EPOCHS_MNIST if dataset == "mnist" else EPOCHS_CIFAR

            print("\n=============================================")
            print(
```

```
            f" RUN: dataset={dataset} | attn={attn} | rpe={rpe} |␣
↪epochs={epochs}"
        )
        print("================================================\n")

        run_training_RPE(
            dataset=dataset,
            attn_type=attn,
            rpe_type=rpe,
            epochs=epochs,
            lr=1e-3,
            batch_size=128,
        )
```

```
================================================
 RUN: dataset=cifar10 | attn=favor+ | rpe=rope | epochs=60
================================================

Using device: cuda
[cifar10][attn=favor+][rpe=rope] Epoch 01: train loss=1.7617, train acc=0.3482,
test loss=1.5459, test acc=0.4360
[cifar10][attn=favor+][rpe=rope] Epoch 02: train loss=1.5084, train acc=0.4496,
test loss=1.4172, test acc=0.4897
[cifar10][attn=favor+][rpe=rope] Epoch 03: train loss=1.4252, train acc=0.4819,
test loss=1.3808, test acc=0.4993
[cifar10][attn=favor+][rpe=rope] Epoch 04: train loss=1.3712, train acc=0.5050,
test loss=1.3463, test acc=0.5136
[cifar10][attn=favor+][rpe=rope] Epoch 05: train loss=1.3258, train acc=0.5202,
test loss=1.2983, test acc=0.5333
[cifar10][attn=favor+][rpe=rope] Epoch 06: train loss=1.2918, train acc=0.5352,
test loss=1.2651, test acc=0.5410
[cifar10][attn=favor+][rpe=rope] Epoch 07: train loss=1.2570, train acc=0.5462,
test loss=1.2244, test acc=0.5595
[cifar10][attn=favor+][rpe=rope] Epoch 08: train loss=1.2284, train acc=0.5567,
test loss=1.2057, test acc=0.5674
[cifar10][attn=favor+][rpe=rope] Epoch 09: train loss=1.2069, train acc=0.5645,
test loss=1.2067, test acc=0.5626
[cifar10][attn=favor+][rpe=rope] Epoch 10: train loss=1.1761, train acc=0.5765,
test loss=1.1873, test acc=0.5713
[cifar10][attn=favor+][rpe=rope] Epoch 11: train loss=1.1561, train acc=0.5850,
test loss=1.1725, test acc=0.5733
[cifar10][attn=favor+][rpe=rope] Epoch 12: train loss=1.1351, train acc=0.5913,
test loss=1.1532, test acc=0.5883
[cifar10][attn=favor+][rpe=rope] Epoch 13: train loss=1.1187, train acc=0.5960,
test loss=1.1439, test acc=0.5835
[cifar10][attn=favor+][rpe=rope] Epoch 14: train loss=1.1001, train acc=0.6055,
test loss=1.1339, test acc=0.5916
```

[cifar10][attn=favor+][rpe=rope] Epoch 15: train loss=1.0792, train acc=0.6121, test loss=1.1250, test acc=0.5942
[cifar10][attn=favor+][rpe=rope] Epoch 16: train loss=1.0602, train acc=0.6193, test loss=1.1314, test acc=0.5959
[cifar10][attn=favor+][rpe=rope] Epoch 17: train loss=1.0463, train acc=0.6241, test loss=1.1051, test acc=0.6082
[cifar10][attn=favor+][rpe=rope] Epoch 18: train loss=1.0349, train acc=0.6301, test loss=1.0875, test acc=0.6066
[cifar10][attn=favor+][rpe=rope] Epoch 19: train loss=1.0145, train acc=0.6350, test loss=1.0847, test acc=0.6163
[cifar10][attn=favor+][rpe=rope] Epoch 20: train loss=1.0016, train acc=0.6410, test loss=1.0742, test acc=0.6177
[cifar10][attn=favor+][rpe=rope] Epoch 21: train loss=0.9826, train acc=0.6480, test loss=1.0680, test acc=0.6200
[cifar10][attn=favor+][rpe=rope] Epoch 22: train loss=0.9706, train acc=0.6517, test loss=1.0631, test acc=0.6218
[cifar10][attn=favor+][rpe=rope] Epoch 23: train loss=0.9549, train acc=0.6574, test loss=1.0407, test acc=0.6282
[cifar10][attn=favor+][rpe=rope] Epoch 24: train loss=0.9448, train acc=0.6612, test loss=1.0302, test acc=0.6333
[cifar10][attn=favor+][rpe=rope] Epoch 25: train loss=0.9283, train acc=0.6671, test loss=1.0431, test acc=0.6328
[cifar10][attn=favor+][rpe=rope] Epoch 26: train loss=0.9155, train acc=0.6733, test loss=1.0566, test acc=0.6291
[cifar10][attn=favor+][rpe=rope] Epoch 27: train loss=0.9020, train acc=0.6750, test loss=1.0271, test acc=0.6417
[cifar10][attn=favor+][rpe=rope] Epoch 28: train loss=0.8888, train acc=0.6830, test loss=1.0161, test acc=0.6427
[cifar10][attn=favor+][rpe=rope] Epoch 29: train loss=0.8687, train acc=0.6890, test loss=1.0163, test acc=0.6441
[cifar10][attn=favor+][rpe=rope] Epoch 30: train loss=0.8601, train acc=0.6918, test loss=1.0010, test acc=0.6524
[cifar10][attn=favor+][rpe=rope] Epoch 31: train loss=0.8474, train acc=0.6953, test loss=1.0129, test acc=0.6525
[cifar10][attn=favor+][rpe=rope] Epoch 32: train loss=0.8415, train acc=0.7007, test loss=0.9991, test acc=0.6543
[cifar10][attn=favor+][rpe=rope] Epoch 33: train loss=0.8193, train acc=0.7059, test loss=0.9913, test acc=0.6554
[cifar10][attn=favor+][rpe=rope] Epoch 34: train loss=0.8164, train acc=0.7086, test loss=0.9984, test acc=0.6589
[cifar10][attn=favor+][rpe=rope] Epoch 35: train loss=0.8020, train acc=0.7137, test loss=0.9930, test acc=0.6577
[cifar10][attn=favor+][rpe=rope] Epoch 36: train loss=0.7857, train acc=0.7188, test loss=0.9878, test acc=0.6549
[cifar10][attn=favor+][rpe=rope] Epoch 37: train loss=0.7809, train acc=0.7205, test loss=0.9835, test acc=0.6599
[cifar10][attn=favor+][rpe=rope] Epoch 38: train loss=0.7666, train acc=0.7245, test loss=0.9755, test acc=0.6651

[cifar10][attn=favor+][rpe=rope] Epoch 39: train loss=0.7617, train acc=0.7273, test loss=0.9922, test acc=0.6609
[cifar10][attn=favor+][rpe=rope] Epoch 40: train loss=0.7491, train acc=0.7296, test loss=0.9946, test acc=0.6614
[cifar10][attn=favor+][rpe=rope] Epoch 41: train loss=0.7385, train acc=0.7355, test loss=0.9861, test acc=0.6646
[cifar10][attn=favor+][rpe=rope] Epoch 42: train loss=0.7300, train acc=0.7371, test loss=0.9847, test acc=0.6673
[cifar10][attn=favor+][rpe=rope] Epoch 43: train loss=0.7219, train acc=0.7415, test loss=0.9569, test acc=0.6778
[cifar10][attn=favor+][rpe=rope] Epoch 44: train loss=0.7121, train acc=0.7440, test loss=0.9801, test acc=0.6697
[cifar10][attn=favor+][rpe=rope] Epoch 45: train loss=0.7006, train acc=0.7490, test loss=0.9859, test acc=0.6735
[cifar10][attn=favor+][rpe=rope] Epoch 46: train loss=0.6916, train acc=0.7498, test loss=0.9735, test acc=0.6692
[cifar10][attn=favor+][rpe=rope] Epoch 47: train loss=0.6876, train acc=0.7538, test loss=0.9718, test acc=0.6749
[cifar10][attn=favor+][rpe=rope] Epoch 48: train loss=0.6774, train acc=0.7565, test loss=0.9885, test acc=0.6717
[cifar10][attn=favor+][rpe=rope] Epoch 49: train loss=0.6738, train acc=0.7571, test loss=0.9939, test acc=0.6719
[cifar10][attn=favor+][rpe=rope] Epoch 50: train loss=0.6622, train acc=0.7635, test loss=0.9869, test acc=0.6751
[cifar10][attn=favor+][rpe=rope] Epoch 51: train loss=0.6495, train acc=0.7668, test loss=0.9629, test acc=0.6744
[cifar10][attn=favor+][rpe=rope] Epoch 52: train loss=0.6462, train acc=0.7682, test loss=0.9896, test acc=0.6782
[cifar10][attn=favor+][rpe=rope] Epoch 53: train loss=0.6416, train acc=0.7680, test loss=0.9801, test acc=0.6776
[cifar10][attn=favor+][rpe=rope] Epoch 54: train loss=0.6322, train acc=0.7737, test loss=0.9892, test acc=0.6762
[cifar10][attn=favor+][rpe=rope] Epoch 55: train loss=0.6264, train acc=0.7751, test loss=0.9837, test acc=0.6792
[cifar10][attn=favor+][rpe=rope] Epoch 56: train loss=0.6193, train acc=0.7763, test loss=0.9995, test acc=0.6740
[cifar10][attn=favor+][rpe=rope] Epoch 57: train loss=0.6120, train acc=0.7810, test loss=0.9997, test acc=0.6799
[cifar10][attn=favor+][rpe=rope] Epoch 58: train loss=0.5991, train acc=0.7835, test loss=1.0301, test acc=0.6736
[cifar10][attn=favor+][rpe=rope] Epoch 59: train loss=0.6003, train acc=0.7834, test loss=0.9978, test acc=0.6804